

Scalable Federated and Parallel Algorithms for Unique Column

Combination Discovery

Supervisor: Dr. Atakan Aral

2026S SE Masterseminar

Department of Computer Science, University of Vienna



universität
wien

Author: Muhammad Raza Ali
Master's Student: 12248246, University of Vienna

30 June, 2026

Abstract

Minimal hitting set enumeration arises in a variety of data profiling tasks, particularly for discovering minimal unique column combinations (UCCs) in databases. The report proposes a thesis in which the algorithm by Bläsius, Friedrich, Lischoid, Meeks, and Schirneck [1] is adapted for efficient execution in a federated setting. Where clients stream encoded local constraints to a coordinating hub. The hub aggregates these constraints into partial hypergraph, enumerates global minimal hitting sets using branch-parallel extension-oracle pruning, the approach is designed to operate iteratively on smaller partitions or groups of data, allowing scalable analysis of large datasets.

1 Introduction

Unique Column Combination (UCC) discovery is an important data profiling task in relational databases. A UCC is a set of attributes whose combined values uniquely identify each tuple in a table. Discovering such combinations is useful for candidate-key discovery, schema analysis, data quality assessment, and database optimization. However, UCC discovery is computationally expensive because the number of possible column combinations increases exponentially with the number of attributes. This seminar report focuses on scalable UCC discovery through minimal hitting set enumeration. In particular, it investigates whether the hitting set enumeration algorithm by *Bläsius, Friedrich, Lischoid, Meeks, and Schirneck* [1], which is based on decision-tree enumeration and extension-oracle pruning, can be adapted to a federated and parallel setting. This is challenging because, in many practical scenarios, data is distributed across multiple nodes and raw tuples cannot be transferred to a central server. To address this problem, the proposed research studies a *coordinator-client architecture* [6]. Each client keeps its local data private and sends only encoded local constraints to a coordinating hub. The hub aggregates these constraints and derives global minimal hitting sets. At the same time, the enumeration process is accelerated through multi-core parallel computation on the *ALMA* [2] cluster at the University of Vienna. The overall goal is to combine federated aggregation with parallel enumeration in order to improve scalability while preserving the correctness of UCC discovery.

1.1 Algorithmic Background

Hypergraph hitting set problems have long been studied for applications in combinatorics and database theory. In the specific context of data profiling, hitting sets capture the notion of *unique column combinations* (UCCs) by representing non-uniqueness evidence as hyperedges, such that any set of columns intersecting all hyperedges yields a candidate key. This seminar report builds on the hitting set enumeration algorithm recent work by Bläsius *et al.* [1]. Their approach uses a decision-tree enumeration strategy together with an *extension oracle*, which helps prune branches that cannot lead to minimal solutions. This makes the algorithm suitable as a foundation for scalable UCC discovery. The proposed thesis investigates how this algorithmic idea can be adapted to a federated and parallel setting. Instead of centralizing the full dataset, the data is distributed across multiple client nodes. Each client keeps its raw tuples local and sends only encoded local constraints to a coordinating hub. The hub aggregates these constraints into a global hypergraph and enumerates global minimal hitting sets. In addition, parallel execution is used to process different branches of the enumeration tree simultaneously, on the *ALMA* [2] many-core cluster. The main research question is whether this combination of federated constraint aggregation and parallel enumeration can improve scalability while preserving the correctness of global UCC discovery.

1.2 Preliminaries

Hypergraphs: A *hypergraph* is a generalization of a standard graph in which an edge can contain any number of vertices (rather than exactly two). Formally, a hypergraph H is defined by a pair $(V, \mathcal{H})$ where V is the set of vertices and $\mathcal{H} \subseteq 2^V$ is the set of hyperedges.

For example, in **Figure 1**, the vertex set is $V = \{a, b, c, d, e\}$, and we have four hyperedges:

$$\{a, b, c\}, \quad \{a, b, c, d, e\}, \quad \{d, e\}, \quad \{b, c, d\}.$$

Unlike a standard (undirected) graph where each edge would just link two vertices, here an edge can link three, four, or even all five of the vertices at once.

Minimal Hitting Sets: A *hitting set* for $H = (V, E)$ is a subset $S \subseteq V$ that intersects every hyperedge $e \in E$. Equivalently, for each $e \in E$, we require $S \cap e \neq \emptyset$. A hitting set S is *minimal* if no proper subset of S is itself a hitting set. Removing any single element from a minimal hitting set would leave at least one hyperedge uncovered.

Example. Figure 1 shows a hypergraph with $V = \{a, b, c, d, e\}$ and four edges: $\{a, b, c\}$, $\{a, b, c, d, e\}$, $\{d, e\}$, $\{b, c, d\}$. The set $\{c, e\}$ is a minimal hitting set, as it intersects all edges, but neither $\{c\}$ nor $\{e\}$ alone is sufficient.

Unique Column Combinations in Databases: In database systems, a *unique column combination* (UCC) is a set of attributes (columns) such that every row in the table can be uniquely identified by at least one column in that set. Figure 2 illustrates a small dataset with columns **Age**, **Name**, **Address**, **City**, and **Area Code**. If two rows share the same values for **Name** and **Age** (and have different values everywhere else), then $\{\text{Name}, \text{Age}\}$ alone would *not* uniquely distinguish those rows in general. But this pair shows that every UCC must contain either the **Address**, **City**, or **Area Code**. A proper subset of columns that fails to distinguish *at least one* pair of rows is useless as a UCC.

To reframe this as a hitting set problem, we note:

- **Vertices** correspond to column names.
- **Hyperedges** correspond to *difference sets*, which contain all attributes where the values in the two rows differ.

A minimal hitting set then represents a minimal set of columns that, collectively, breaks *every* collision, thus uniquely identifying each row. If we removed any column from this set, at least one pair of rows would remain indistinguishable. These examples demonstrate how fundamental notions of hypergraphs and hitting sets carry directly over to finding minimal UCCs.

1.3 Algorithm Overview

The goal is to compute all *minimal hitting sets*, ensuring that no subset of a solution is also a valid hitting set. A recursive decision tree-based approach is used to construct minimal hitting sets efficiently. The decision tree explores possible solutions by selecting elements iteratively, while an *extension oracle* prunes non-minimal or redundant branches. The extension oracle decides for a pair (X, Y) of disjoint sets of vertices, whether X can be extended to a minimal hitting set without using any vertex from Y .

Each branch in the decision tree corresponds to a candidate hitting set, and it *prunes non-minimal or redundant branches*.

The algorithm operates recursively as follows:

1. **Initialize** the decision tree with an empty set.
2. **Apply the extension oracle** to determine if the set is contained in a minimal hitting set.
3. **Expand** a set of elements that might form a hitting set iteratively into a decision tree to find minimal hitting sets.

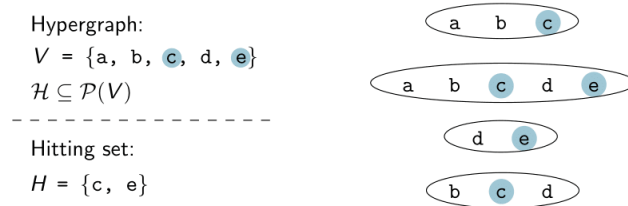


Figure 1: A hypergraph with vertices $\{a, b, c, d, e\}$. Each oval represents a hyperedge. The set $\{c, e\}$ hits every hyperedge and is minimal. (Figure by M. Schirneck.)

Age	Name	Address	City	Area Code	
47	Mustermann, Max	Mittelstraße 125	Potsdam	D-14467	Add City AC
47	Mustermann, Max	W Broadway 400	San Diego	US-CA-92101	Age Name Add City AC
76	Doe, John	South Street 8	London	UK-W1K	Age Name
90	Nightingale, Florence	South Street 8	London	UK-W1K	Age City AC
25	Menigmand, Morten	Trøjburgvej 24	Aarhus	DK-8200	
33	Doe, John	South Street 8	Philadelphia	US-PA-19145	

Figure 2: A table with attributes like **Age**, **Name**, **Address**, **City**, and **Area Code**, and its corresponding hypergraph representation. Potential unique column combinations map to minimal hitting sets. (Figure by M. Schirneck.)

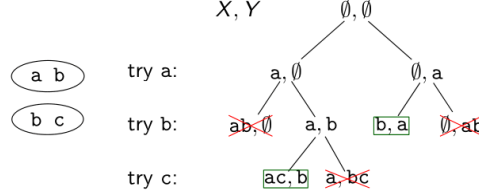


Figure 3: Decision tree for hitting set enumeration with extension oracle pruning. Green nodes indicate minimal solutions, while red nodes are pruned. (Figure by M. Schirneck.)

4. **If minimal, store the hitting set;** if the set is not yet a minimal hitting set, but contained in one, then recurse and go back to Step 2 *Expand*; otherwise, prune the branch.
5. **Recurse on remaining elements** until all minimal hitting sets are found.

This recursive structure ensures that no redundant solutions are generated, as computing the extension oracle itself is very expensive. The tradeoff between pruning and the cost of the oracle is not always clear.

Base Case: The algorithm outputs X as a minimal hitting set when X intersects all hyperedges in Y , terminating recursion, with condition $X \cap e \neq \emptyset \forall e \in Y$, to the action (output and termination). **Recursive Expansion:** Each element in the vertex set V is selected one by one and added to X , forming a new candidate hitting set. **Pruning via Extension Oracle:** The function checks whether the newly constructed X remains minimal. **Backtracking:** If minimality fails, the branch is pruned, avoiding unnecessary computations.

2 Background

UCC discovery has been studied through several algorithmic directions, ranging from centralized Apriori-style search to scalable lattice traversal and hitting-set-based enumeration. These approaches are relevant for this seminar report because they show how existing methods handle the large search space of column combinations and where they are limited when data is distributed across multiple nodes.

2.1 Centralized UCC Discovery

Early UCC discovery methods mainly follow two strategies. Row-based approaches, such as GORDIAN, analyze tuple patterns and build in-memory structures to identify non-unique column combinations. Column-based approaches, such as Apriori-style algorithms, explore the search space level by level, starting with single columns and then testing larger combinations. Abedjan and Naumann [7] improve this idea with HCA, which uses candidate pruning, value-distribution histograms, and discovered dependencies to reduce the number of combinations that must be checked. These methods are useful as classical baselines, but they are designed for centralized settings where the full dataset is available on one machine. This becomes a limitation for large datasets and federated scenarios, where raw tuples may be distributed across different owners and cannot be moved to a central server.

2.2 Scalable Lattice-Based Discovery

DUCC, proposed by Heise *et al.* [8], improves scalability by treating UCC discovery as a traversal of the lattice of column combinations. It prunes supersets of already discovered uniques and subsets of known non-uniques. DUCC combines greedy search with random walks near the boundary between unique and non-unique combinations. This makes the search more output-sensitive, because the runtime depends more strongly on the number of actual minimal UCCs than on the complete search space.

DUCC also supports parallel and distributed execution, which makes it important for this research. However, it is still a lattice-traversal method rather than a hitting-set enumeration algorithm. Therefore, the ideas about parallel workers and scalable search are useful as inspiration, but its algorithmic structure cannot be directly reused for the proposed hitting-set-based federated approach.

2.3 Hitting-Set-Based UCC Discovery

The main algorithmic foundation of this seminar report is the hitting set enumeration algorithm by Bläsius, Friedrich, Lischeid, Meeks, and Schirneck [1]. In this approach, UCC discovery is reduced to minimal hitting set enumeration over a hypergraph of difference sets. Each hyperedge represents columns in which two tuples differ. A set of columns is unique if it intersects all such hyperedges.

The algorithm uses a decision-tree enumeration process and an extension oracle to prune branches that cannot lead to minimal solutions. This makes it attractive for scalable UCC discovery, because different branches of the enumeration tree can potentially be processed in parallel. However, the original method assumes that the difference-set hypergraph is already available centrally. It does not address federated data owners, local constraints, or distributed validation.

2.4 Partial-Information Hitting Set Discovery

HPIValid, proposed by Birnick, *et al.* [9], addresses the cost of constructing the complete difference-set hypergraph. Instead of comparing all tuple pairs at once, it starts from partial information, enumerates candidate hitting sets, validates them against the table, and adds missing difference sets when validation fails. This allows enumeration to continue from refined information instead of restarting from the beginning.

This idea is especially relevant for the proposed thesis. In a federated setting, the coordinator may also start with incomplete global information, because each client only contributes encoded local constraints. The same general pattern can therefore be reused: aggregate partial constraints, enumerate candidates, validate them locally, and refine the global hypergraph when missing information is found. The main difference is that HPIValid works in a centralized setting, while the proposed thesis applies this idea across distributed clients and combines it with parallel enumeration. Previous work provides important building blocks for scalable UCC discovery, but no method fully combines federated constraint aggregation, global minimal hitting set enumeration, and parallel execution. This gap motivates the research direction of this seminar report.

3 Preliminary Investigations

Before the thesis, two practical implementations were carried out to examine the main parts of the proposed research separately. The first practical focused on parallelizing hitting set enumeration on a shared dataset, while the second practical investigated a federated clients-server environment in which data is distributed across multiple nodes. Together, these practicals provide the starting point for the proposed thesis, which aims to combine parallel enumeration with federated UCC discovery. Both reuse the same algorithmic core (Lischeid’s [4] C++ implementation sequential of the Bläsius *et al.* [1] enumeration, packaged as the enumhyp tool) and the same family of benchmark datasets from the Hasso Plattner Institut [3].

3.1 Parallel Enumeration on ALMA

The *first practical* [5] studied how the hitting set enumeration algorithm by Bläsius *et al.* [1] can be parallelized on ALMA [2], the many-core cluster at the University of Vienna. The implementation used a

master-slave MPI architecture. The master maintained a global queue of enumeration tasks, while worker nodes processed different branches of the decision tree. Each task represented a partial enumeration state, and workers could return new branches to the master when further splitting was possible.

Number of experiments showed that parallel execution is useful mainly for larger datasets. For very small datasets, the sequential version was faster because the enumeration finished quickly and the communication overhead between master and workers dominated the runtime. For larger datasets, however, parallelization provided clear speedups because different branches of the search tree could be evaluated simultaneously. This result shows that branch-level parallelism is a suitable strategy for the Bläsius *et al.* [1] enumeration approach, but only when the computational work per task is large enough to justify the communication cost.

This practical is important for the thesis because it shows that the decision-tree structure of the hitting set algorithm can be divided into independent tasks. Therefore, a coordinating hub can potentially distribute enumeration branches across multiple cores and nodes when the global problem is large enough.

3.2 Federated Client-Server Aggregation

The *second practical* [6] investigated UCC discovery in a federated setting. Instead of processing the full dataset on signal machine environment, the data was split across several client nodes. Each client kept its raw data local and executed a local generation and enumeration pipeline. The clients then sent only derived column-index information to a central server. The server collected the client outputs and applied aggregation strategies to derive global results.

Two mainly aggregation strategies were tested. The first strategy used intersection, keeping only UCCs that appeared in the output of every client. This was computationally cheap and worked well in the practical experiments where the clients held partitions of the same schema. Drawback would be if any client doesn't have set in different UCCs datasets. The second strategy, called MinUnion, combined local UCCs from different clients and then removed non-minimal supersets. This strategy was more general, but it became expensive as the number of clients and local UCCs increased.

The practical showed that federated execution can reduce runtime for larger datasets because the local computation is distributed across clients. However, it also revealed that communication, container startup, and aggregation overhead can dominate for small datasets. Therefore, as in the parallel practical, the benefit depends on the size and complexity of the input.

4 Proposed Architecture

The proposed thesis investigates a scalable architecture for discovering minimal Unique Column Combinations (UCCs) in federated and parallel environments. The main objective is to adapt hitting-set-based UCC discovery to a setting where data is distributed across several client nodes and raw tuples cannot be transferred to a central server. At the same time, the thesis aims to reduce the runtime of minimal hitting set enumeration by using parallel computation on multiple cores and compute nodes.

The proposed system combines two levels of distribution. The first level is the federated client level, where each client owns a local data partition and performs local preprocessing. The second level is the core-computation level, where a coordinating hub distributes enumeration tasks across multiple worker cores and nodes. Together, these two levels form a Client $\times$ Core architecture: clients contribute encoded local information, while cores accelerate the global enumeration process.

4.1 Client-Side Processing

Figure 4 illustrates the role of a client node. Each client stores its own raw relational data locally. The raw tuples are not sent to the hub. Instead, the client performs a local generation step in which the raw data is transformed into encoded or dummy-coded constraints. These constraints represent non-uniqueness evidence, for example in the form of local difference sets (encoded column-index combinations).

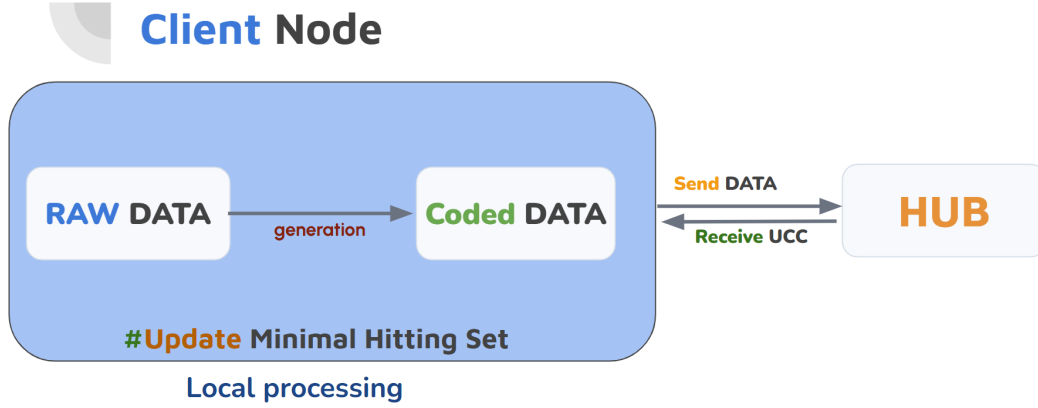


Figure 4: Client node: raw data remains local, while encoded constraints are sent to the hub for global UCC discovery. After each iteration of aggregation, global minimal hitting set update on each client node.

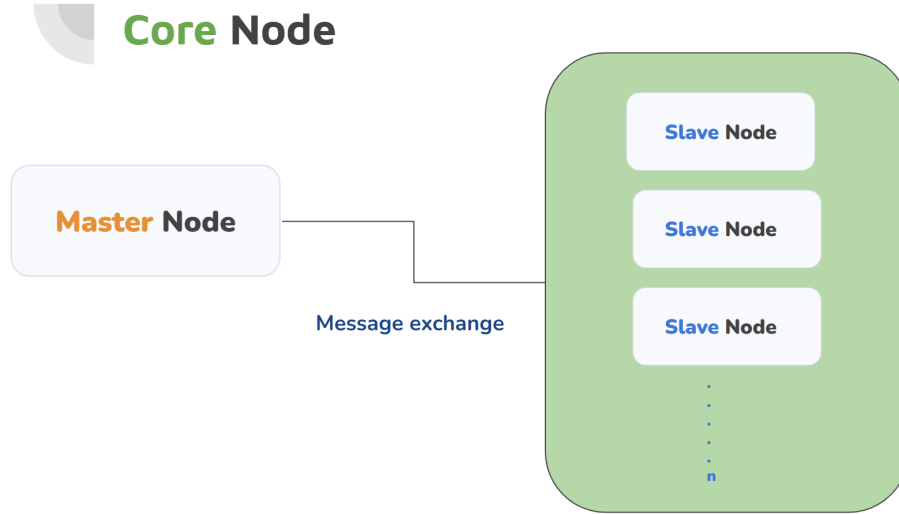


Figure 5: Core-level computation: a master-slave architecture using the Message passing interface (MPI) to parallelize the hitting set enumeration algorithm. The design focuses on load-balanced task distribution while maintaining minimal communication overhead.

The purpose of this step is to preserve data locality while still providing enough information for global UCC discovery. A client may also participate in validation: when the hub produces a candidate global minimal hitting set, the client can check whether this candidate is valid on its local data. If the candidate fails locally, the client returns the missing encoded constraint that explains the failure. In this way, clients do not expose raw attribute values, but they still help the hub refine the global search space.

4.2 Core-Level Parallel Enumeration

Figure 5 shows the parallel computation part of the architecture. After receiving encoded constraints from the clients, the hub constructs a global partial hypergraph. Minimal UCC discovery is then treated as a minimal hitting set enumeration problem over this hypergraph.

The enumeration process follows the decision-tree structure of the algorithm by Bläsius *et al.* [1]. At each step, the algorithm branches over candidate columns and uses an extension oracle to prune branches that cannot lead to minimal solutions. This structure is suitable for parallel execution. Instead of running the enumeration sequentially on a single node because different branches of the enumeration tree can be processed



Framework

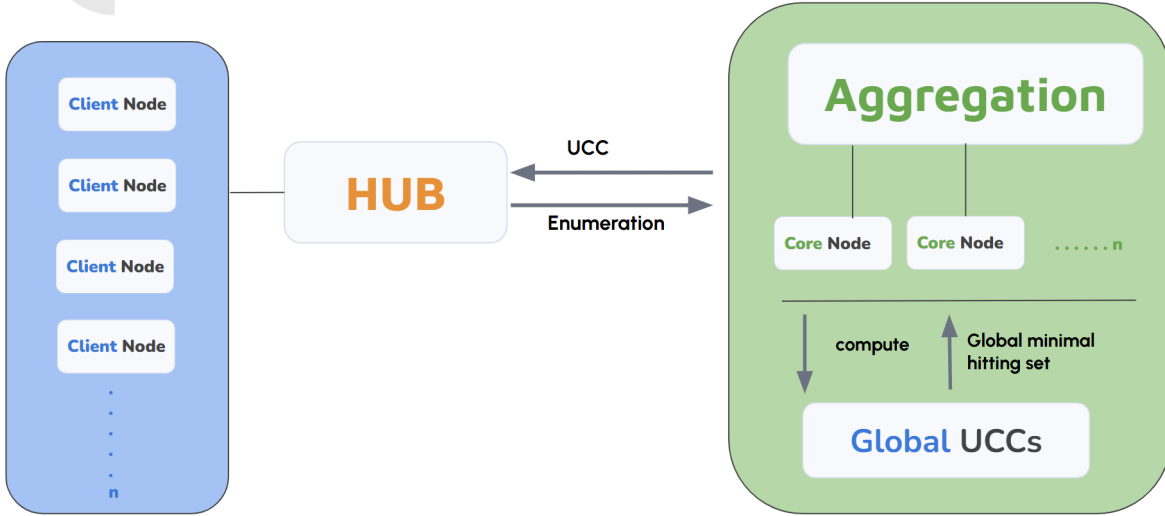


Figure 6: Proposed framework: clients provide encoded constraints, the hub aggregates them, and worker cores compute global minimal UCCs through parallel hitting set enumeration.

independently.

In the proposed architecture, the hub acts as a coordinator. It maintains a queue of enumeration tasks and assigns independent branches to worker cores of nodes. Each worker evaluates its assigned branch, applies the extension oracle, and returns either valid candidate solutions or newly generated subproblems. This design allows the system to exploit multi-core computation, for example on the *ALMA* [2] cluster at the University of Vienna.

4.3 Proposed Framework

The complete framework is shown in Figure 6. Multiple client nodes send encoded local constraints to the hub. The hub aggregates these constraints into a global partial hypergraph and starts the enumeration of global minimal hitting sets. Since enumeration is computationally expensive, the hub distributes different branches of the enumeration tree across multiple worker cores of compute nodes.

The key difference from a simple federated aggregation approach is that clients do not independently finalize local UCCs and then combine the results by union or intersection. In the earlier prototype, aggregation was effectively performed in two stages: first at the client level, where each client generated and enumerated local minimal hitting sets, and then at the global level, where the hub tried to aggregate the already minimized local results, such approach is expensive in terms of time and cost. The proposed thesis performs minimality checking at the global level. The hub computes global candidate hitting sets from the combined constraints and, sends these candidates back to the clients for local verification in validation mode.

We assume, a candidate will valid on all clients as the algorithm is tested on multiple practical, it is accepted as a global minimal UCC. If a candidate fails on one or more clients, the responsible clients return additional encoded constraints that explain the failure. The hub adds these constraints to the global partial hypergraph and continues enumeration from the refined state instead of restarting the entire process. This creates an iterative workflow: collect constraints, enumerate candidates, validate locally, refine the hypergraph, and continue with the next batch or iteration.

In the proposed framework, a candidate hitting set is accepted as a global minimal UCC only if it is valid on all clients. Based on the preliminary practical evaluations, we expect most candidates generated from the aggregated by enumeration constraints to satisfy this condition. However, if a candidate fails on one or more

clients, the responsible clients return additional encoded constraints that explain the failure. The hub then adds these constraints to the global partial hypergraph and continues enumeration from the refined state instead of restarting the entire process. This creates an iterative workflow: collect constraints, enumerate candidates, validate locally, refine the hypergraph, and continue with the next batch of iteration.

4.4 Expected Contributions

The expected contribution of the thesis is a scalable framework for discovering minimal Unique Column Combinations (UCCs) in federated and parallel environments. The proposed approach combines two ideas: federated constraint aggregation and branch-parallel minimal hitting set enumeration. Instead of centralizing raw tuples, each client keeps its local data private and sends only encoded local constraints to a coordinating hub. The hub then aggregates these constraints into a global partial hypergraph and computes global minimal hitting sets.

A second contribution is the parallelization of the enumeration process. The thesis builds on the hitting set enumeration algorithm by *Bläsius, Friedrich, Lischeid, Meeks, and Schirneck* [1], which uses decision-tree enumeration and pruning via an extension oracle. Since different branches of the enumeration tree can be processed independently, the proposed framework distributes expensive enumeration tasks across multiple worker cores and compute nodes. The implementation is run and tested on *ALMA* [2], a hardware resource of *University of Vienna*. A "Heterogeneous Many-Core Cluster" comprising a frontend and multiple heterogeneous computational nodes, all based on Supermicro SuperServer systems. While the cluster itself has four computational nodes, the experiments utilize 16 virtual nodes created by dividing the physical nodes into multiple virtual instances. Each virtual node is connected via a network with dedicated memory. Transferring data from node to node is expensive in time (during program execution).

The thesis will also address a correctness problem observed in the preliminary federated prototype. Combining already minimized local UCCs by union or intersection is not sufficient in general, because local minimality does not guarantee global minimality. Therefore, the proposed architecture shifts minimality checking to the global level. Candidate global hitting sets are validated against the clients, and if validation fails, the clients return additional encoded constraints. This allows the hub to refine the global hypergraph and continue enumeration without restarting from scratch.

The evaluation will compare the proposed approach with a centralized baseline and with the preliminary practical implementations. The main evaluation criteria will be correctness of the discovered global UCCs, runtime performance, communication overhead, scalability with the number of clients, and scalability with the number of worker cores.

5 Conclusion

This seminar report presented a thesis proposal for scalable UCC discovery in federated and parallel environments. The main challenge is to compute globally minimal Unique Column Combinations without centralizing raw data, while still keeping the enumeration process efficient for large datasets.

The reviewed literature and preliminary practicals show that both parallel enumeration and federated processing are promising, but they also reveal an important limitation: combining already minimized local UCCs by union or intersection does not generally guarantee globally minimal results. Therefore, the proposed thesis shifts the focus from aggregating local solutions to aggregating encoded local constraints.

The proposed architecture uses a coordinating hub to build a global partial hypergraph from client-provided constraints, enumerate global minimal hitting sets using branch-parallel computation, and validate candidates against local client data. This design aims to preserve correctness while improving scalability through distributed processing and parallel execution.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Atakan Aral, for his guidance, constructive feedback, and support throughout the development of this thesis proposal. His comments helped me refine the federated system design, clarify the aggregation problem, and identify the correctness gap between local and global minimal hitting sets. I also thank Dr. Martin Schirneck for providing datasets and supporting materials, and for his work on UCC discovery that forms the foundation of this project, including the method by Bläsius, Friedrich, Lischeid, Meeks, and Schirneck [1].

References

- [1] T. Bläsius, T. Friedrich, J. Lischeid, K. Meeks, M. Schirneck, *Efficiently Enumerating Hitting Sets of Hypergraphs Arising in Data Profiling*. Journal of Computer and System Sciences, 2022 (previous version in ALENEX 2019). <https://arxiv.org/abs/1805.01310>
- [2] ALMA, *Computer Science - Hardware Resources, University of Vienna*. <https://www.par.univie.ac.at/index.php?goto=hardware>.
- [3] Hasso-Plattner-Institut, *Enumeration in Data Profiling* <https://hpi.de/friedrich/research/enumdat.html>
- [4] *Special thanks go to Julius Lischeid C++ code on GitHub* <https://github.com/goodefroi/enumhyp>
- [5] Practical I, “Parallelizing Hitting Set Enumeration Algorithms for Data Profiling.” *Praktikum Informatik I report*, supervised by M. Schirneck, University of Vienna, 14 February 2025.
- [6] Practical II, *Federated Data Profiling and Unique Column Combination Discovery*. “Praktikum Informatik II report, supervised by A. Aral, University of Vienna, 29 January 2026.
- [7] Abedjan & Naumann 2011, Z. Abedjan, F. Naumann. “Advancing the Discovery of Unique Column Combinations.” *CIKM 2011*, Glasgow, UK.
- [8] DUCC, A. Heise, J.-A. Quiané-Ruiz, Z. Abedjan, A. Jentzsch, F. Naumann. “Scalable Discovery of Unique Column Combinations.” *Proceedings of the VLDB Endowment*, 7(4), 2013/2014 (invited to VLDB 2014).
- [9] HPIValid, Birnick, T. Bläsius, T. Friedrich, F. Naumann, T. Papenbrock, M. Schirneck. “Hitting Set Enumeration with Partial Information for Unique Column Combination Discovery.” *Proceedings of the VLDB Endowment*, 13(11), 2020.